



Content-Based Image Retrieval Project

Master: TAID, anul 2

Echipa: Boşcan Tiberiu-Ioan, Ion
Florentin-Georgian, Stancovici Marian

Cuprins

Obiectiv

Setul de date: CBIR_dataset

Preprocesare și normalizare

Augmentare de date: rol și exemple

Construirea perechilor (Siamese Dataset)

DataLoader și batch-uri

Arhitectura modelului: Embedding Network

Arhitectura siameză: principiul de funcționare

Funcția de cost: Contrastive Loss

Antrenarea modelului

Extracția embeddings pentru baza de date (Indexing)

Similarity search: mecanismul CBIR

Observații, limitări și direcții de îmbunătățire

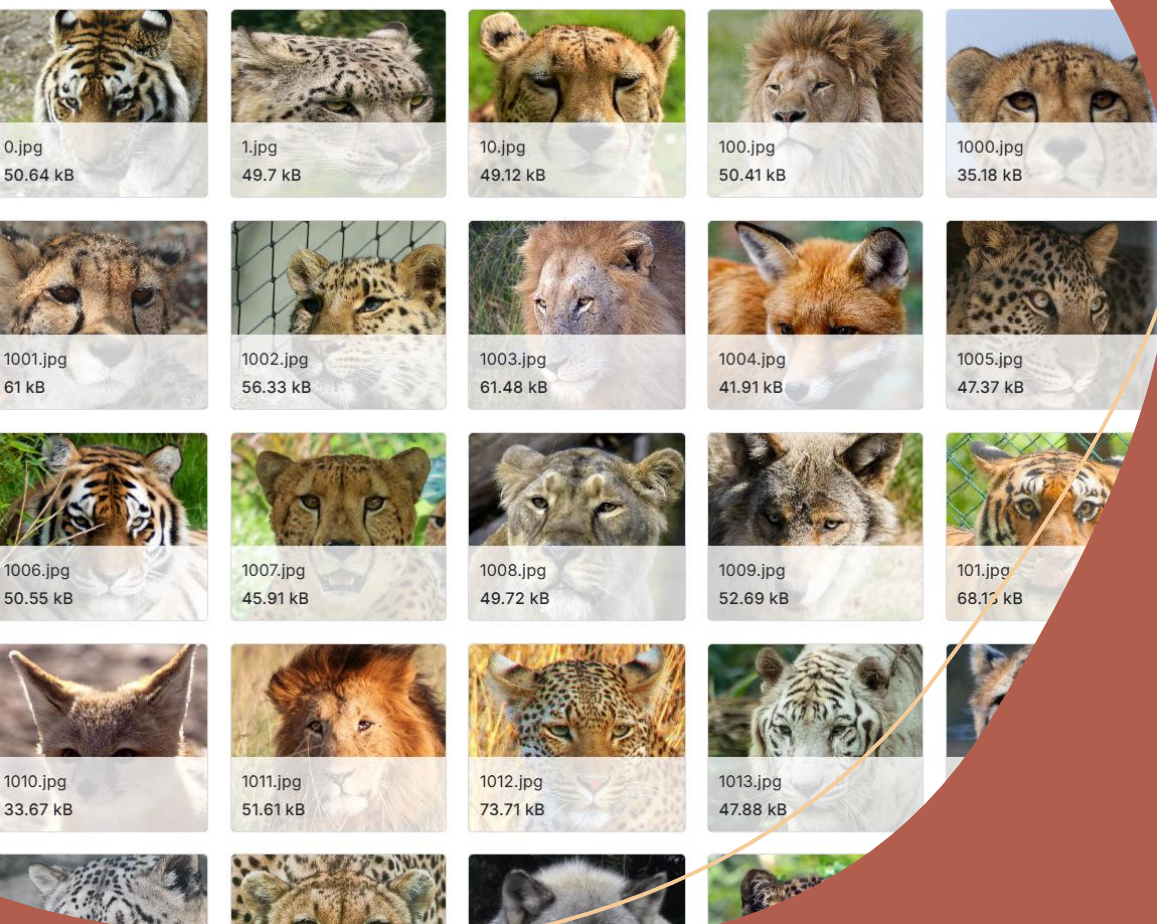
Concluzii

Obiectiv

- Implementarea unui sistem **Content-Based Image Retrieval (CBIR)** bazat pe **învățare metrică** (metric learning)
- Obiectiv: învățarea unui spațiu de reprezentare (embedding) în care imaginile similare sunt apropiate
- Abordare: arhitectură **siameză** cu backbone CNN pre-antrenat și funcție de cost contrastivă
- Rezultat: căutare Top-K pe baza distanței dintre embeddings

About this directory

This directory does not have a description yet.



Setul de date: CBIR_dataset

- Set de imagini organizat **fără etichete explicite/clase** (fișiere .jpg într-un singur director)
- Implicație: nu se pot construi perechi pozitive/negative pe baza claselor; necesară o schemă self-supervised
- Preprocesare: colectarea tuturor căilor în DataFrame și împărțire train/validation (80/20)
- Considerații: volum de date, diversitate vizuală, rolul validării în monitorizarea convergenței

Preprocesare și normalizare

Redimensionare la o rezoluție fixă (224×224) pentru compatibilitate cu backbone CNN

Conversia imaginii în tensor (format numeric) și normalizarea canalelor RGB

Normalizarea (mean/std ImageNet) pentru stabilitatea antrenării și compatibilitate cu modele pre-antrenate

Separarea transformărilor: pipeline specific train vs. pipeline specific validation

Augmentare de date: rol și exemple

- Scop: creșterea acurateții la transformări realiste ale imaginii (invarianțe)
- Augmentări folosite: random crop, flip orizontal, jitter de culoare (luminozitate/contrast/saturație)
- Efect: modelul învață „conținutul” și nu memorizează o singură versiune exactă a imaginii
- Justificare: augmentarea este baza perechilor pozitive în regim self-supervised

```
train_transform = T.Compose([
    T.Resize(256),
    T.RandomResizedCrop(224, scale=(0.8, 1.0)),
    T.RandomHorizontalFlip(),
    T.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1),
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])

val_transform = T.Compose([
    T.Resize((224, 224)),
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
])
```

```

class SiameseDataset(Dataset):
    def __init__(self, df, transform=None):
        self.df = df.reset_index(drop=True)
        self.transform = transform

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        img1_path = self.df.loc[idx, "path"]

        # 1 = pereche pozitivă (aceeași imagine), 0 = negativă (imagine diferită)
        same_image = random.randint(0, 1)

        if same_image == 1:
            img2_path = img1_path
            target = 1.0
        else:
            idx2 = idx
            while idx2 == idx:
                idx2 = random.randint(0, len(self.df) - 1)
            img2_path = self.df.loc[idx2, "path"]
            target = 0.0

        img1 = Image.open(img1_path).convert("RGB")
        img2 = Image.open(img2_path).convert("RGB")

        if self.transform is not None:
            img1 = self.transform(img1)
            img2 = self.transform(img2)

        target = torch.tensor([target], dtype=torch.float32)
        return img1, img2, target

```

Construirea perechilor (Siamese Dataset)

- Fiecare eșantion returnează un triplet: (img1, img2, target)
- Pereche pozitivă: două augmentări ale aceleiași imagini (target=1)
- Pereche negativă: imaginea curentă vs. o imagine aleatoare diferită (target=0)
- Beneficiu: formularea problemei ca „învățare a similarității” fără etichete manuale

DataLoader și batch-uri

- DataLoader construiește mini-batch-uri: [B, 3, 224, 224] pentru img1 și img2
- Shuffle pe train pentru a evita ordine deterministă și corelații nedorite
- Avantaj: antrenare stabilă, paralelizare și utilizare eficientă a GPU
- Obiectiv: optimizarea loss-ului pe batch-uri de perechi pozitive și negative

```
batch_size = 32

train_dataset = SiameseDataset(train_df, transform=train_transform)
val_dataset = SiameseDataset(val_df, transform=val_transform)

train_loader = DataLoader(
    train_dataset, batch_size=batch_size, shuffle=True, num_workers=2
)
val_loader = DataLoader(
    val_dataset, batch_size=batch_size, shuffle=False, num_workers=2
)

len(train_loader), len(val_loader)
```

```
class EmbeddingNet(nn.Module):
    def __init__(self, embedding_dim=128):
        super().__init__()
        |
        try:
            weights = models.ResNet18_Weights.IMAGENET1K_V1
            self.backbone = models.resnet18(weights=weights)
        except AttributeError:
            self.backbone = models.resnet18(pretrained=True)

        in_features = self.backbone.fc.in_features
        self.backbone.fc = nn.Linear(in_features, embedding_dim)

    def forward(self, x):
        x = self.backbone(x)
        x = F.normalize(x, p=2, dim=1)
        return x
```

Arhitectura modelului: Embedding Network

- Backbone: ResNet18 pre-antrenat, utilizat ca extractor general de trăsături vizuale
- Adaptare: înlocuirea stratului final de clasificare cu un strat care produce un embedding (ex. 128D)
- Normalizare L2 a embedding-ului pentru comparații consistente între imagini
- Output: vector numeric ce rezumă informația vizuală relevantă

Arhitectura siameză: principiul de funcționare

- Două intrări (img1, img2) trec prin **aceeași** rețea de embedding (greutăți partajate)
- Output: două embeddings (z1, z2) în același spațiu latent
- Motiv: comparabilitate directă între reprezentări și antrenare orientată spre similaritate
- Interpretare: distanța dintre z1 și z2 devine scorul de similaritate

```
from torch.nn import Module

from classes import EmbeddingNet

▼ class SiameseNet(Module):
    def __init__(self, embedding_dim=128):
        super().__init__()
        self.embedding_net = EmbeddingNet(embedding_dim)

    def forward(self, x1, x2):
        z1 = self.embedding_net(x1)
        z2 = self.embedding_net(x2)
        return z1, z2
```

```

class ContrastiveLoss(nn.Module):
    def __init__(self, margin=1.0):
        super().__init__()
        self.margin = margin

    def forward(self, z1, z2, y):
        # z1, z2: [B, D], y: [B, 1] (1 = similar, 0 = diferit)
        distances = F.pairwise_distance(z1, z2) # [B]
        y = y.view(-1) # [B]

        loss_pos = y * distances.pow(2)
        loss_neg = (1 - y) * F.relu(self.margin - distances).pow(2)

        loss = (loss_pos + loss_neg).mean()
        return loss

```

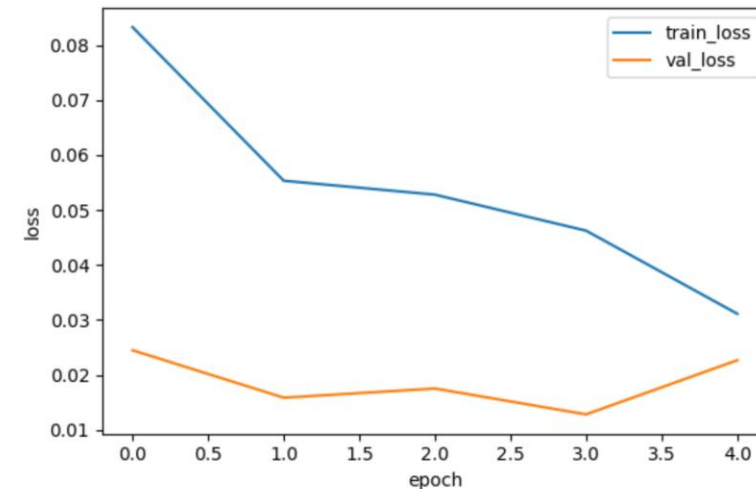
Funcția de loss: Contrastive Loss

- Măsurarea distanței dintre embeddings (ex. L2 / Euclidean)
- Pentru perechi pozitive: penalizare când distanța este mare (împinge embeddings aproape)
- Pentru perechi negative: penalizare când distanța este sub o margine (margin) (împinge embeddings departe)
- Rolul marginii: stabilește separarea minimă dorită între exemple diferite

Antrenarea modelului

- Optimizator: Adam, rată de învățare mică (ex. $1e-4$) pentru stabilitate
- Procedură: forward $\rightarrow$ loss $\rightarrow$ backward $\rightarrow$ update parametri, repetat pe epoci
- Monitorizare: loss pe train și loss pe validation pentru semne de overfitting/underfitting
- Rezultat așteptat: scăderea loss-ului și creșterea separării între pozitive și negative

```
Embeddings path: /home/tiberiu/Documents/TAID/SCAV/Similar-Images-Recognition/models/cbir_embe
Paths CSV: /home/tiberiu/Documents/TAID/SCAV/Similar-Images-Recognition/models/cbir_image_path
Running short training: samples=400, epochs=5, device=cuda
Epoch 1/5 | train_loss=0.0833 val_loss=0.0245 time=13.53s
Epoch 2/5 | train_loss=0.0553 val_loss=0.0158 time=13.52s
Epoch 3/5 | train_loss=0.0528 val_loss=0.0175 time=13.41s
Epoch 4/5 | train_loss=0.0462 val_loss=0.0128 time=13.31s
Epoch 5/5 | train_loss=0.0311 val_loss=0.0226 time=13.39s
```



```
Training run complete. Summary:
train_loss: [0.0832560065290928, 0.05531499199569225, 0.05278315395116806, 0.0462303023785352, 0.03111111111111111]
```

```
num_epochs = 5

for epoch in range(num_epochs):
    train_loss = train_one_epoch(model, train_loader, optimizer, criterion, device)
    val_loss = eval_one_epoch(model, val_loader, criterion, device)

    print(f"Epoch [{epoch+1}/{num_epochs}] "
          f"- train loss: {train_loss:.4f} | val loss: {val_loss:.4f}")
```

```
Epoch [1/5] - train loss: 0.0631 | val loss: 0.0170
Epoch [2/5] - train loss: 0.0397 | val loss: 0.0162
Epoch [3/5] - train loss: 0.0344 | val loss: 0.0120
Epoch [4/5] - train loss: 0.0299 | val loss: 0.0108
Epoch [5/5] - train loss: 0.0305 | val loss: 0.0101
```

Extracția embeddings pentru baza de date (Indexing)

- După antrenare: se calculează embeddings pentru toate imaginile din baza de date
- Se obține o matrice $[N, D]$ (N imagini, D dimensiunea embedding-ului)
- Stocare: embedding -uri salvate pentru a evita recalculări și pentru interogări rapide
- Interpretare: fiecare rând reprezintă „semnătura vizuală” a unei imagini

```
@torch.no_grad()
def compute_embeddings(model, df, transform, device, batch_size=64):
    model.eval()
    all_embeddings = []

    for start in range(0, len(df), batch_size):
        batch_paths = df["path"].iloc[start:start+batch_size].tolist()
        imgs = []
        for p in batch_paths:
            img = Image.open(p).convert("RGB")
            imgs.append(transform(img))
        imgs = torch.stack(imgs).to(device)

        z = model.embedding_net(imgs) # [B, D]
        all_embeddings.append(z.cpu())

    all_embeddings = torch.cat(all_embeddings, dim=0) # [N, D]
    return all_embeddings

|
full_df = pd.concat([train_df, val_df], ignore_index=True)
db_embeddings = compute_embeddings(model, full_df, val_transform, device, batch_size=64)
db_embeddings.shape
```

```
17]: torch.Size([4738, 128])
```

Similarity search: mecanismul CBIR

- Pentru o imagine query: se calculează embedding-ul acesteia
- Se calculează distanțe față de toate embedding-urile din index (vectorizat)
- Se sortează după distanță și se selectează Top-K cele mai apropiate imagini

```
# Cell 16 - Test: afișează imagini similare pentru un index random
# =====

random_idx = random.randint(0, len(full_df) - 1)
print("Query index:", random_idx, " | path:", full_df.loc[random_idx, "path"])

show_similar(random_idx, top_k=5)
```

Query index: 244 | path: /kaggle/input/cbir-dataset/dataset/4194.jpg



Observații, limitări și direcții de îmbunătățire

Limitare: absența etichetelor → negativele aleatoare pot fi uneori „false negatives”

Similaritatea învățată poate fi influențată de textură/culori, nu întotdeauna de semantică

Îmbunătățiri: strategii de hard negatives, loss modern (InfoNCE), model mai puternic (ResNet50/ViT)

Concluzii

A fost implementat un sistem CBIR complet, bazat pe învățarea unei reprezentări numerice pentru imagini și căutare Top-K prin distanțe.

Arhitectura siameză a permis antrenarea în regim self-supervised, folosind perechi pozitive/negative generate prin augmentări și eșantionare aleatoare.

Indexarea embedding-urilor a separat etapa de „prelucrare offline” de etapa de „interogare”, reducând costul de calcul la momentul căutării.

Rezultatele vizuale indică faptul că spațiul de embedding capturează trăsături relevante pentru similaritate în cadrul setului de date utilizat.

Bibliografie

- **CBIR_dataset (Kaggle)** – *Content Based Image Retrieval Dataset*
<https://www.kaggle.com/datasets/theaayushbajaj/cbir-dataset>
- https://docs.pytorch.org/tutorials/beginner/basics/data_tutorial.html
- <https://docs.pytorch.org/vision/main/models/generated/torchvision.models.resnet18.html>
- https://www.researchgate.net/publication/4246277_Dimensionality_Reduction_by_Learning_an_Invariant_Mapping
- chrome-extension://efaidnbmnnnibpcajpcglclefindmkaj/https://www.cs.cmu.edu/~rsalakhu/papers/oneshot1.pdf